PRÁCTICAS EVALUADAS 1 Y 2

Recuperación de contraseña

Java Swing, patrón DAO y almacenamiento seguro con SHA-256 y salt

Estudiante: Jason Azofeifa Flores

Fecha: octubre de 2026

Tecnología: Java 17 o superior, Swing (JFrame)

Pruebas automáticas: 160 de 160 aprobadas

Resumen de la entrega

Este documento reúne las dos prácticas, la evidencia de su funcionamiento y el código principal. Ambas comparten el mismo diseño visual: una columna de marca a la izquierda y el formulario sobre fondo cálido a la derecha, con componentes Swing dibujados a medida (campos redondeados, botones, medidor de fuerza y mensajes de error bajo cada campo).

Carpeta	Contenido
Practica1/	Ventana FrmOlvido, método recuperarContrasena(usuario, correo) en UsuarioDAO y validaciones de formulario.
Practica2/	Regla de negocio: IAutenticacion, UsuarioDAO y UsuarioDAOImpl con SHA-256 + salt aleatorio, formulario de nueva contraseña e inicio de sesión conectado al DAO.
Documento/	Este informe en Word y PDF, y la evidencia (capturas y resultados de pruebas).

Cada práctica trae su .jar listo (java -jar), los scripts ejecutar.bat y ejecutar.sh, el código fuente en src/ y las pruebas en pruebas/. La cuenta de prueba de la Práctica 2 es demo@correo.com con contraseña Demo1234.

Práctica 1: Ventana "Olvidó mi contraseña"

1. Objetivo

Implementar una ventana independiente para recuperar la contraseña con Java Swing, declarar y implementar el método en la capa DAO (solo estructura) y validar los campos del formulario.

2. Cumplimiento de requerimientos

Requerimiento	Cómo se cumple	Estado
JFrame llamado FrmOlvido	Clase vista.FrmOlvido extends JFrame	CUMPLE
Ventana centrada	setLocationRelativeTo(null)	CUMPLE
Resizable = false	setResizable(false)	CUMPLE
Cerrar sin finalizar el sistema	setDefaultCloseOperation(DISPOSE_ON_CLOSE)	CUMPLE
Componentes para la recuperación	Campos de usuario y correo, botón Recuperar y enlace Cancelar	CUMPLE
Método en la interfaz UsuarioDAO	boolean recuperarContrasena(String usuario, String correo)	CUMPLE
Método implementado en la clase DAO	UsuarioDAOImpl (estructura, sin base de datos)	CUMPLE
Campos obligatorios y solo espacios	Validador.estaVacio() sobre el texto recortado	CUMPLE
Mensajes informativos	Mensaje bajo cada campo con el problema concreto	CUMPLE
Formato de correo	Validador.correoValido() con expresión regular estricta	CUMPLE
No continuar con errores	hayErrores() corta el flujo antes de llamar al DAO	CUMPLE

3. Evidencia visual



Figura 1. Inicio de sesión con el enlace de recuperación



Figura 2. FrmOlvido al abrirse



Figura 3. Formulario enviado vacío



Figura 4. Correo con formato inválido



Figura 5. Solicitud procesada

4. Pruebas ejecutadas

Pruebas de lógica (sin ventanas): 27/27 pruebas aprobadas. Pruebas sobre las ventanas reales: 28/28 pruebas aprobadas.

N.°	Prueba	Resultado
1	Correo válido aceptado: a@b.co	APROBADA
2	Correo válido aceptado: nombre.apellido@dominio.com	APROBADA
3	Correo válido aceptado: x_y+z@sub.dominio.cr	APROBADA
4	Correo válido aceptado: A1@mail-2.org	APROBADA
5	Correo inválido rechazado: "correo"	APROBADA
6	Correo inválido rechazado: "correo@"	APROBADA
7	Correo inválido rechazado: "@dominio.com"	APROBADA
8	Correo inválido rechazado: "a@b"	APROBADA

N.°	Prueba	Resultado
9	Correo inválido rechazado: "a b@c.com"	APROBADA
10	Correo inválido rechazado: "a@b.."	APROBADA
11	Correo inválido rechazado: "a@b.c"	APROBADA
12	Correo inválido rechazado: "a@..b.com"	APROBADA
13	Correo inválido rechazado: "a@x..com"	APROBADA
14	Correo inválido rechazado: "a@-x.com"	APROBADA
15	Correo inválido rechazado: "a@x-.com"	APROBADA
16	Correo inválido rechazado: "..@x.com"	APROBADA
17	Correo inválido rechazado: "a..b@x.com"	APROBADA
18	Correo inválido rechazado: ".a@x.com"	APROBADA
19	Correo inválido rechazado: "a.@x.com"	APROBADA
20	Correo inválido rechazado: ""	APROBADA
21	Correo null rechazado	APROBADA
22	estaVacio: null, vacío y solo espacios	APROBADA
23	estaVacio: texto con contenido	APROBADA
24	UsuarioDAO declara recuperarContrasena(String, String)	APROBADA
25	UsuarioDAOImpl implementa UsuarioDAO	APROBADA
26	DAO: recuperarContrasena(usuario, correo) devuelve true	APROBADA
27	DAO: con null devuelve false	APROBADA

N.°	Prueba	Resultado
1	FrmOlvido es un JFrame	APROBADA
2	Aparece centrada en la pantalla	APROBADA
3	Resizable = false	APROBADA
4	Cierre = DISPOSE_ON_CLOSE	APROBADA
5	Formulario vacío: marca los dos campos -> "Escriba su usuario." / "Escriba su correo electrónico."	APROBADA
6	La ventana no cambia de tamaño al mostrar errores	APROBADA
7	Con errores no se llama al DAO	APROBADA
8	Usuario solo con espacios se rechaza -> Escriba su usuario.	APROBADA
9	Correo solo con espacios se rechaza -> Escriba su correo electrónico.	APROBADA
10	Correo con formato inválido "correo" -> El correo no tiene un formato válido.	APROBADA
11	Correo con formato inválido "correo@" -> El correo no tiene un formato válido.	APROBADA
12	Correo con formato inválido "@dominio.com" -> El correo no tiene un formato válido.	APROBADA
13	Correo con formato inválido "a@b" -> El correo no tiene un formato válido.	APROBADA
14	Correo con formato inválido "a b@c.com" -> El correo no tiene un formato válido.	APROBADA
15	Correo con formato inválido "a@..b.com" -> El correo no tiene un formato válido.	APROBADA

N.°	Prueba	Resultado
16	Correo con formato inválido "a@x..com" -> El correo no tiene un formato válido.	APROBADA
17	Correo con formato inválido "a@-x.com" -> El correo no tiene un formato válido.	APROBADA
18	La ventana sigue abierta y no se llamó al DAO	APROBADA
19	El error se borra al corregir el campo	APROBADA
20	Datos válidos: se llama al DAO una vez con usuario y correo	APROBADA
21	Datos válidos: aparece la pantalla de confirmación	APROBADA
22	Cerrar con la X cierra solo esta ventana (la JVM sigue viva)	APROBADA
23	La tecla Esc cierra la ventana	APROBADA
24	FrmLogin se muestra y no se puede redimensionar	APROBADA
25	Login vacío -> Escriba su usuario.	APROBADA
26	Login sin contraseña -> Escriba su contraseña.	APROBADA
27	Pulsar el enlace varias veces abre una sola ventana FrmOlvido	APROBADA
28	Al cerrar FrmOlvido el login principal sigue abierto	APROBADA

Práctica 2: Regla de negocio de recuperación

1. Objetivo

Declarar en la interfaz de autenticación el método de recuperación de contraseña e implementarlo en el DAO guardando la contraseña con SHA-256 y un salt aleatorio, nunca en texto plano.

2. Diseño de la solución

- IAutenticacion: declara boolean recuperarContrasena(String correo, String nuevaContrasena).
- UsuarioDAO: extiende IAutenticacion y agrega verificarContrasena para el inicio de sesión.
- UsuarioDAOImpl: busca el usuario por correo, genera un salt nuevo, calcula el hash y reemplaza su registro en usuarios.txt con el formato correo;salt;hash.
- SeguridadClave: salt de 16 bytes con SecureRandom y SHA-256(salt + contraseña), comparación en tiempo constante.
- Validador: formato de correo y fuerza de la contraseña (de 1 a 4).
- FrmOlvido y FrmLogin: interfaz; el login ya consulta el DAO, de modo que se puede comprobar de punta a punta que el cambio funcionó.

Flujo: el usuario escribe su correo y la nueva contraseña (dos veces); el formulario valida todos los campos a la vez; el DAO identifica al usuario, cifra la contraseña y guarda salt y hash. Si el correo no existe, retorna false y la ventana lo informa. No se envían correos ni códigos de verificación, tal como indica la práctica.

3. Cumplimiento de requerimientos

Requerimiento	Cómo se cumple	Estado
Método declarado en la interfaz	IAutenticacion (heredado por UsuarioDAO)	CUMPLE
Implementado en la clase DAO	UsuarioDAOImpl.recuperarContrasena	CUMPLE
No guardar en texto plano	Solo se persisten salt y hash	CUMPLE
SHA-256 con salt	MessageDigest SHA-256 sobre salt + contraseña	CUMPLE
Salt aleatorio por contraseña	SecureRandom, 16 bytes, nuevo en cada cambio	CUMPLE
Salt almacenado junto al hash	Registro correo;salt;hash	CUMPLE
Recibe datos para identificar y registrar	Parámetros correo y nuevaContrasena	CUMPLE
Sin correos ni códigos de verificación	No se implementan (fuera del alcance)	CUMPLE

4. Evidencia visual



Figura 6. FrmOlvido al abrirse



Figura 7. Errores por campo y medidor "Débil"



Figura 8. Contraseña fuerte



Figura 9. Contraseña actualizada



Figura 10. Correo sin cuenta



Figura 11. Entrada con la contraseña nueva

Registro de usuarios.txt antes y después de recuperar la contraseña. Solo aparecen el salt y el hash:

```
Registro antes:   demo@correo.com;gZ7AJVn/1kmKJ+n5vrAW5Q==;XTuPtmft7jIvrhq68LRd3NnF3pkPS0gnQDv4PlaPkzE=
Registro después: demo@correo.com;o0p1eIcZ5B4JUDM1b/ZwXg==;Cl9+/Dwpd3AvaUpOx2hhv7AyKHelLjJEhrG8Qf6kIVI=
```

5. Pruebas ejecutadas

Pruebas de lógica (sin ventanas): 58/58 pruebas aprobadas. Pruebas sobre las ventanas reales: 47/47 pruebas aprobadas. Incluyen una comprobación independiente que recalcula SHA-256(salt +

contraseña) y la compara con el hash guardado, 40 cambios de contraseña simultáneos y un recorrido completo: cambiar la contraseña desde el enlace del login y volver a entrar con ella.

N.°	Prueba	Resultado
1	Correo válido aceptado: a@b.co	APROBADA
2	Correo válido aceptado: nombre.apellido@dominio.com	APROBADA
3	Correo válido aceptado: x_y+z@sub.dominio.cr	APROBADA
4	Correo válido aceptado: A1@mail-2.org	APROBADA
5	Correo inválido rechazado: "correo"	APROBADA
6	Correo inválido rechazado: "correo@"	APROBADA
7	Correo inválido rechazado: "@dominio.com"	APROBADA
8	Correo inválido rechazado: "a@b"	APROBADA
9	Correo inválido rechazado: "a b@c.com"	APROBADA
10	Correo inválido rechazado: "a@b.."	APROBADA
11	Correo inválido rechazado: "a@b.c"	APROBADA
12	Correo inválido rechazado: "a@..b.com"	APROBADA
13	Correo inválido rechazado: "a@x..com"	APROBADA
14	Correo inválido rechazado: "a@-x.com"	APROBADA
15	Correo inválido rechazado: "a@x-.com"	APROBADA
16	Correo inválido rechazado: "..@x.com"	APROBADA
17	Correo inválido rechazado: "a..b@x.com"	APROBADA
18	Correo inválido rechazado: ".a@x.com"	APROBADA
19	Correo inválido rechazado: "a.@x.com"	APROBADA
20	Correo inválido rechazado: ""	APROBADA
21	Correo inválido rechazado: "a@b.com."	APROBADA
22	Correo null rechazado	APROBADA
23	Correo de más de 254 caracteres rechazado	APROBADA
24	estaVacio: null, vacío y solo espacios	APROBADA
25	estaVacio: texto con contenido	APROBADA
26	Fuerza de vacío = 0	APROBADA
27	Fuerza de contraseña corta = 1	APROBADA
28	Fuerza de Demo1234 = 2	APROBADA
29	Fuerza de MiClave2026x = 3	APROBADA
30	Fuerza de MiClave#2026x = 4	APROBADA
31	Salt de 16 bytes	APROBADA
32	Hash de 32 bytes (SHA-256)	APROBADA
33	Hash == SHA-256(salt + contraseña) calculado aparte	APROBADA
34	Mismo salt y contraseña dan el mismo hash	APROBADA
35	Otro salt cambia el hash	APROBADA

N.°	Prueba	Resultado
36	coincide() acepta la correcta	APROBADA
37	coincide() rechaza la incorrecta	APROBADA
38	2000 salts generados, todos distintos	APROBADA
39	Se crea el archivo con la cuenta demo	APROBADA
40	Cuenta demo entra con Demo1234	APROBADA
41	recuperarContrasena devuelve true si el correo existe	APROBADA
42	Formato correo;salt;hash	APROBADA
43	El archivo no contiene la contraseña en texto plano	APROBADA
44	Salt nuevo en cada cambio	APROBADA
45	Hash distinto tras el cambio	APROBADA
46	Clave nueva entra	APROBADA
47	Clave anterior ya no entra	APROBADA
48	No queda archivo temporal tras guardar	APROBADA
49	Correo se compara sin importar mayúsculas ni espacios	APROBADA
50	Correo inexistente devuelve false	APROBADA
51	Correo null devuelve false	APROBADA
52	Contraseña null devuelve false	APROBADA
53	Contraseña en blanco devuelve false y no cambia nada	APROBADA
54	verificarContrasena con null devuelve false	APROBADA
55	Contraseña con tildes y símbolos	APROBADA
56	Con varios usuarios solo cambia el solicitado	APROBADA
57	Archivo con líneas dañadas: no lanza error y devuelve false	APROBADA
58	40 cambios simultáneos: el archivo queda con 1 línea íntegra	APROBADA

N.°	Prueba	Resultado
1	FrmOlvido es un JFrame	APROBADA
2	Aparece centrada en la pantalla	APROBADA
3	Resizable = false	APROBADA
4	Cierre = DISPOSE_ON_CLOSE	APROBADA
5	Formulario vacío: marca los tres campos a la vez	APROBADA
6	Mensajes de campo vacío: "Escriba su correo electrónico." / "Escriba la nueva contraseña." / "Repita la contraseña."	APROBADA
7	La ventana no cambia de tamaño al mostrar errores	APROBADA
8	Correo solo con espacios se rechaza como vacío	APROBADA
9	Correo con formato inválido "correo" -> El correo no tiene un formato válido.	APROBADA
10	Correo con formato inválido "correo@" -> El correo no tiene un formato válido.	APROBADA
11	Correo con formato inválido "@dominio.com" -> El correo no tiene un formato válido.	APROBADA

N.°	Prueba	Resultado
12	Correo con formato inválido "a@b" -> El correo no tiene un formato válido.	APROBADA
13	Correo con formato inválido "a b@c.com" -> El correo no tiene un formato válido.	APROBADA
14	Correo con formato inválido "a@..b.com" -> El correo no tiene un formato válido.	APROBADA
15	Correo con formato inválido "a@x..com" -> El correo no tiene un formato válido.	APROBADA
16	Correo con formato inválido "a@-x.com" -> El correo no tiene un formato válido.	APROBADA
17	Correo con formato inválido "..@x.com" -> El correo no tiene un formato válido.	APROBADA
18	Contraseña de solo espacios -> La contraseña no puede ser solo espacios.	APROBADA
19	Contraseña menor a 8 caracteres -> Use al menos 8 caracteres.	APROBADA
20	Sin confirmación -> Repita la contraseña.	APROBADA
21	Confirmación distinta -> Las contraseñas no coinciden.	APROBADA
22	Con errores el hash guardado no cambia	APROBADA
23	La ventana sigue abierta mientras hay errores	APROBADA
24	El error de un campo se borra al corregirlo	APROBADA
25	Medidor: "abc" nivel 1	APROBADA
26	Medidor: "Demo1234" nivel 2	APROBADA
27	Medidor: "MiClave#2026x" nivel 4	APROBADA
28	Medidor: campo vacío nivel 0	APROBADA
29	El ojito muestra la contraseña	APROBADA
30	El ojito vuelve a ocultarla	APROBADA
31	Correo inexistente -> No hay ninguna cuenta con ese correo.	APROBADA
32	Flujo exitoso: aparece la pantalla de confirmación	APROBADA
33	Flujo exitoso: la clave nueva entra y la anterior no	APROBADA
34	El archivo no contiene la contraseña en texto plano	APROBADA
35	El hash cambió	APROBADA
36	Cerrar con la X cierra solo esta ventana (la JVM sigue viva)	APROBADA
37	La tecla Esc cierra la ventana	APROBADA
38	FrmLogin se muestra centrada y sin redimensionar	APROBADA
39	Login vacío -> Escriba su correo electrónico.	APROBADA
40	Login sin contraseña -> Escriba su contraseña.	APROBADA
41	Login con contraseña incorrecta -> Correo o contraseña incorrectos.	APROBADA
42	Login con la cuenta demo entra -> Acceso correcto. Sesión iniciada.	APROBADA
43	Recorrido completo: el cambio desde FrmOlvido se aplica	APROBADA
44	Recorrido completo: la contraseña vieja ya no entra	APROBADA
45	Recorrido completo: la contraseña nueva entra -> Acceso correcto. Sesión iniciada.	APROBADA
46	Pulsar el enlace varias veces abre una sola ventana FrmOlvido	APROBADA
47	Al cerrar FrmOlvido el login principal sigue abierto	APROBADA

Revisión y mejoras

Fallos encontrados y corregidos

Problema	Corrección
La validación de correo aceptaba entradas como a@..b.com, a@x..com o a@-x.com.	Nueva expresión que exige etiquetas de dominio válidas y rechaza puntos dobles; cubierta con 17 casos negativos.
Se podían abrir varias ventanas FrmOlvido a la vez desde el login.	El login conserva la ventana abierta y la trae al frente.
La ventana cambiaba de tamaño al mostrar un error.	El espacio de cada mensaje está reservado; el tamaño no cambia (hay una prueba para eso).
usuarios.txt se escribía directamente: un corte a mitad de la escritura lo dejaba dañado.	Se escribe a un archivo temporal y se renombra de forma atómica.
El correo distinguía mayúsculas y espacios al buscar al usuario.	Se normaliza (recortar y pasar a minúsculas) antes de comparar.
Cuadros de diálogo de Java con botones en inglés (OK).	Se eliminaron: los avisos y la confirmación están dentro de la propia ventana.
Solo se mostraba el primer error del formulario.	Se muestran todos los errores a la vez, cada uno bajo su campo, y el foco va al primero.
El login no estaba conectado al DAO.	Ahora verifica contra el hash guardado, lo que permite probar el cambio de punta a punta.

Decisiones de diseño

- Paquetes separados (app, dao, util, vista, vista.ui) para que cada clase tenga una sola responsabilidad.
- El DAO recibe la ruta del archivo por constructor, lo que permite probarlo con archivos temporales sin tocar los datos reales.
- FrmOlvido recibe el UsuarioDAO por constructor; así las pruebas pueden sustituirlo por uno de prueba.
- Las contraseñas se comparan en tiempo constante (MessageDigest.isEqual).

Limitaciones conocidas

- Se usa un archivo de texto porque la práctica no pide base de datos; en un sistema real iría una tabla.
- Un único SHA-256 con salt cumple lo pedido, pero para producción se recomienda un algoritmo lento y pensado para contraseñas, como PBKDF2, bcrypt o Argon2.
- Como no se envía correo ni código de verificación (el enunciado lo excluye), cualquiera que conozca un correo podría cambiar su contraseña. Es una simplificación válida para esta práctica.

Anexo: código fuente principal

Se incluyen las clases de la capa DAO, la lógica de seguridad y la ventana de recuperación. El resto del código (componentes visuales y pruebas) está en las carpetas src/ y pruebas/ del proyecto.

Práctica 1

Practica1/src/dao/UsuarioDAO.java

```
package dao;

public interface UsuarioDAO {

    boolean recuperarContrasena(String usuario, String correo);
}
```

Practica1/src/dao/UsuarioDAOImpl.java

```
package dao;

public class UsuarioDAOImpl implements UsuarioDAO {

    @Override
    public boolean recuperarContrasena(String usuario, String correo) {
        // En esta práctica solo se pide la estructura del método; la consulta
        // a la base de datos y el envío del correo no forman parte del alcance.
        return usuario != null && correo != null;
    }
}
```

Practica1/src/util/Validador.java

```
package util;

import java.util.regex.Pattern;

public final class Validador {

    public static final int MIN_CLAVE = 8;
    private static final int MAX_CORREO = 254;
    private static final String[] ETIQUETAS = {"", "Débil", "Aceptable", "Buena", "Fuerte"};

    // local@dominio.tld sin puntos dobles ni guiones al inicio o final de cada etiqueta
    private static final Pattern CORREO = Pattern.compile(
            "^[A-Za-z0-9]([A-Za-z0-9._%+-]*[A-Za-z0-9])?"
            + "@([A-Za-z0-9]([A-Za-z0-9-]*[A-Za-z0-9])?\\.)+[A-Za-z]{2,}$");

    private Validador() {
    }

    public static boolean estaVacio(String texto) {
        return texto == null || texto.isBlank();
    }

    public static boolean correoValido(String correo) {
        return correo != null
                && correo.length() <= MAX_CORREO
                && !correo.contains("..")
                && CORREO.matcher(correo).matches();
    }

    /** Devuelve 0 si no hay texto y de 1 (débil) a 4 (fuerte) en los demás casos. */
    public static int fuerza(String clave) {
        if (clave == null || clave.isEmpty()) {
            return 0;
        }
        if (clave.length() < MIN_CLAVE) {
            return 1;
        }
        boolean mayus = clave.chars().anyMatch(Character::isUpperCase);
        boolean minus = clave.chars().anyMatch(Character::isLowerCase);
        boolean digito = clave.chars().anyMatch(Character::isDigit);
```

```
        boolean simbolo = clave.chars().anyMatch(c -> !Character.isLetterOrDigit(c));

        int puntos = 1;
        if (clave.length() >= 12) puntos++;
        if (mayus && minus && digito) puntos++;
        if (simbolo) puntos++;
        return Math.min(puntos, 4);
    }

    public static String etiquetaFuerza(int nivel) {
        return ETIQUETAS[Math.max(0, Math.min(nivel, ETIQUETAS.length - 1))];
    }
}
```

Practica1/src/vista/FrmOlvido.java

```
package vista;

import dao.UsuarioDAO;
import dao.UsuarioDAOImpl;
import util.Validador;
import vista.ui.Boton;
import vista.ui.CampoTexto;
import vista.ui.Enlace;
import vista.ui.PanelMarca;
import vista.ui.Piezas;
import vista.ui.Sello;
import vista.ui.Tema;
import vista.ui.Validable;

import javax.swing.BorderFactory;
import javax.swing.BoxLayout;
import javax.swing.JComponent;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.KeyStroke;
import java.awt.BorderLayout;
import java.awt.CardLayout;
import java.awt.Dimension;
import java.awt.event.KeyEvent;

public class FrmOlvido extends JFrame {

    private static final String PASO_FORMULARIO = "formulario";
    private static final String PASO_LISTO = "listo";

    private final UsuarioDAO usuarioDAO;

    private final CampoTexto txtUsuario = new CampoTexto();
    private final CampoTexto txtCorreo = new CampoTexto();
    private final JLabel errUsuario = Piezas.mensaje();
    private final JLabel errCorreo = Piezas.mensaje();
    private final Boton btnRecuperar = new Boton("Recuperar contraseña", true);

    private final CardLayout pasos = new CardLayout();
    private final JPanel lado = new JPanel(pasos);

    public FrmOlvido() {
        this(new UsuarioDAOImpl());
    }

    public FrmOlvido(UsuarioDAO usuarioDAO) {
        this.usuarioDAO = usuarioDAO;
        armarVentana();
        conectarEventos();
    }

    private void armarVentana() {
        setTitle("Olvidó mi contraseña");
        setIconImage(Tema.icono());
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);
        setResizable(false);

        lado.setBackground(Tema.PAPEL);
        lado.add(crearFormulario(), PASO_FORMULARIO);
        lado.add(crearConfirmacion(), PASO_LISTO);
```

```
        JPanel raiz = new JPanel(new BorderLayout());
        raiz.add(new PanelMarca("Recupere el acceso a su cuenta.",
                "Indique su usuario y el correo con el que se registró para iniciar la recuperación."),
                BorderLayout.WEST);
        raiz.add(lado, BorderLayout.CENTER);
        raiz.setPreferredSize(new Dimension(870, 620));
        setContentPane(raiz);

        pack();
        setLocationRelativeTo(null);
    }

    private JPanel crearFormulario() {
        JPanel p = columna();
        p.add(Piezas.hueco(62));
        p.add(Piezas.titulo("Olvidé mi contraseña"));
        p.add(Piezas.hueco(8));
        p.add(Piezas.parrafo("Complete los datos de su cuenta para continuar."));
        p.add(Piezas.hueco(30));

        p.add(Piezas.etiqueta("Usuario"));
        p.add(Piezas.hueco(6));
        p.add(Piezas.alinear(txtUsuario));
        p.add(errUsuario);
        p.add(Piezas.hueco(10));

        p.add(Piezas.etiqueta("Correo electrónico"));
        p.add(Piezas.hueco(6));
        p.add(Piezas.alinear(txtCorreo));
        p.add(errCorreo);
        p.add(Piezas.hueco(22));

        p.add(Piezas.alinear(btnRecuperar));
        p.add(Piezas.hueco(14));
        p.add(new Enlace("Cancelar", this::dispose));
        return p;
    }

    private JPanel crearConfirmacion() {
        JPanel p = columna();
        p.add(Piezas.hueco(110));
        p.add(new Sello());
        p.add(Piezas.hueco(26));
        p.add(Piezas.titulo("Solicitud procesada"));
        p.add(Piezas.hueco(10));
        p.add(Piezas.parrafo("Su solicitud de recuperación fue procesada."));
        p.add(Piezas.hueco(34));
        Boton volver = new Boton("Volver al inicio", true);
        volver.addActionListener(e -> dispose());
        p.add(Piezas.alinear(volver));
        return p;
    }

    private static JPanel columna() {
        JPanel p = new JPanel();
        p.setLayout(new BoxLayout(p, BoxLayout.Y_AXIS));
        p.setOpaque(false);
        p.setBorder(BorderFactory.createEmptyBorder(44, 52, 36, 52));
        return p;
    }

    private void conectarEventos() {
        btnRecuperar.addActionListener(e -> recuperar());
        getRootPane().setDefaultButton(btnRecuperar);
        getRootPane().registerKeyboardAction(e -> dispose(),
                KeyStroke.getKeyStroke(KeyEvent.VK_ESCAPE, 0), JComponent.WHEN_IN_FOCUSED_WINDOW);

        Piezas.alEscribir(txtUsuario, () -> limpiar(errUsuario, txtUsuario));
        Piezas.alEscribir(txtCorreo, () -> limpiar(errCorreo, txtCorreo));
    }

    private void recuperar() {
        String usuario = txtUsuario.getText().trim();
        String correo = txtCorreo.getText().trim();

        if (!hayErrores(usuario, correo) && usuarioDAO.recuperarContrasena(usuario, correo)) {
            pasos.show(lado, PASO_LISTO);
```

```
        }
    }

    /** Revisa los dos campos juntos y deja el foco en el primero que falle. */
    private boolean hayErrores(String usuario, String correo) {
        limpiar(errUsuario, txtUsuario);
        limpiar(errCorreo, txtCorreo);
        CampoTexto primero = null;

        if (Validador.estaVacio(usuario)) {
            marcar(errUsuario, txtUsuario, "Escriba su usuario.");
            primero = txtUsuario;
        }
        if (Validador.estaVacio(correo)) {
            marcar(errCorreo, txtCorreo, "Escriba su correo electrónico.");
            if (primero == null) primero = txtCorreo;
        } else if (!Validador.correoValido(correo)) {
            marcar(errCorreo, txtCorreo, "El correo no tiene un formato válido.");
            if (primero == null) primero = txtCorreo;
        }

        if (primero != null) {
            primero.requestFocusInWindow();
        }
        return primero != null;
    }

    private static void marcar(JLabel mensaje, Validable campo, String texto) {
        Piezas.decir(mensaje, texto);
        campo.marcarError(true);
    }

    private static void limpiar(JLabel mensaje, Validable campo) {
        Piezas.decir(mensaje, null);
        campo.marcarError(false);
    }
}
```

Práctica 2

Practica2/src/dao/IAutenticacion.java

```
package dao;

public interface IAutenticacion {

    /**
     * Cambia la contraseña del usuario que tenga ese correo.
     * La contraseña se guarda con SHA-256 y salt, nunca en texto plano.
     *
     * @return false si no existe un usuario con ese correo
     */
    boolean recuperarContrasena(String correo, String nuevaContrasena);
}
```

Practica2/src/dao/UsuarioDAO.java

```
package dao;

public interface UsuarioDAO extends IAutenticacion {

    @Override
    boolean recuperarContrasena(String correo, String nuevaContrasena);

    boolean verificarContrasena(String correo, String contrasena);
}
```

Practica2/src/dao/UsuarioDAOImpl.java

```
package dao;

import util.SeguridadClave;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.AtomicMoveNotSupportedException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardCopyOption;
import java.util.ArrayList;
import java.util.List;

/**
 * Guarda los usuarios en un archivo de texto, una línea por usuario:
 * correo;salt;hash
 */
public class UsuarioDAOImpl implements UsuarioDAO {

    private static final String SEPARADOR = ";";
    private static final String CORREO_DEMO = "demo@correo.com";
    private static final String CLAVE_DEMO = "Demo1234";

    private final Path archivo;

    public UsuarioDAOImpl() {
        this(Paths.get("usuarios.txt"));
    }

    public UsuarioDAOImpl(Path archivo) {
        this.archivo = archivo;
        if (!Files.exists(archivo)) {
            crearUsuarioDemo();
        }
    }

    @Override
    public synchronized boolean recuperarContrasena(String correo, String nuevaContrasena) {
        if (correo == null || nuevaContrasena == null || nuevaContrasena.isBlank()) {
            return false;
        }
        String clave = normalizar(correo);
        try {
            List<String> lineas = leer();
```

```
            for (int i = 0; i < lineas.size(); i++) {
                String[] campos = lineas.get(i).split(SEPARADOR);
                if (campos.length == 3 && campos[0].equals(clave)) {
                    String salt = SeguridadClave.nuevoSalt();
                    lineas.set(i, registro(clave, salt, SeguridadClave.hash(nuevaContrasena, salt)));
                    guardar(lineas);
                    return true;
                }
            }
        } catch (IOException e) {
            System.err.println("No se pudo actualizar " + archivo + ": " + e.getMessage());
        }
        return false;
    }

    @Override
    public synchronized boolean verificarContrasena(String correo, String contrasena) {
        if (correo == null || contrasena == null) {
            return false;
        }
        String clave = normalizar(correo);
        try {
            for (String linea : leer()) {
                String[] campos = linea.split(SEPARADOR);
                if (campos.length == 3 && campos[0].equals(clave)) {
                    return SeguridadClave.coincide(contrasena, campos[1], campos[2]);
                }
            }
        } catch (IOException e) {
            System.err.println("No se pudo leer " + archivo + ": " + e.getMessage());
        }
        return false;
    }

    private static String normalizar(String correo) {
        return correo.trim().toLowerCase();
    }

    private static String registro(String correo, String salt, String hash) {
        return String.join(SEPARADOR, correo, salt, hash);
    }

    private List<String> leer() throws IOException {
        List<String> lineas = new ArrayList<>(Files.readAllLines(archivo, StandardCharsets.UTF_8));
        lineas.removeIf(String::isBlank);
        return lineas;
    }

    // Se escribe a un archivo temporal y se renombra, así un corte a mitad de
    // la escritura no deja usuarios.txt a medias.
    private void guardar(List<String> lineas) throws IOException {
        Path temporal = archivo.resolveSibling(archivo.getFileName() + ".tmp");
        Files.write(temporal, lineas, StandardCharsets.UTF_8);
        try {
            Files.move(temporal, archivo, StandardCopyOption.ATOMIC_MOVE,
StandardCopyOption.REPLACE_EXISTING);
        } catch (AtomicMoveNotSupportedException e) {
            Files.move(temporal, archivo, StandardCopyOption.REPLACE_EXISTING);
        }
    }

    private void crearUsuarioDemo() {
        String salt = SeguridadClave.nuevoSalt();
        try {
            guardar(new ArrayList<>(List.of(
                    registro(CORREO_DEMO, salt, SeguridadClave.hash(CLAVE_DEMO, salt)))));
        } catch (IOException e) {
            System.err.println("No se pudo crear " + archivo + ": " + e.getMessage());
        }
    }
}
```

Practica2/src/util/SeguridadClave.java

```
package util;

import java.nio.charset.StandardCharsets;
```

```
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.util.Base64;

/** Salt aleatorio y hash SHA-256(salt + contraseña). Ambos se manejan en Base64. */
public final class SeguridadClave {

    private static final int BYTES_SALT = 16;
    private static final SecureRandom azar = new SecureRandom();

    private SeguridadClave() {
    }

    public static String nuevoSalt() {
        byte[] salt = new byte[BYTES_SALT];
        azar.nextBytes(salt);
        return Base64.getEncoder().encodeToString(salt);
    }

    public static String hash(String clave, String salt) {
        try {
            MessageDigest sha = MessageDigest.getInstance("SHA-256");
            sha.update(Base64.getDecoder().decode(salt));
            return
Base64.getEncoder().encodeToString(sha.digest(clave.getBytes(StandardCharsets.UTF_8)));
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException("Este equipo no tiene SHA-256", e);
        }
    }

    public static boolean coincide(String clave, String salt, String hashGuardado) {
        byte[] calculado = hash(clave, salt).getBytes(StandardCharsets.UTF_8);
        return MessageDigest.isEqual(calculado, hashGuardado.getBytes(StandardCharsets.UTF_8));
    }
}
```

Practica2/src/util/Validador.java

```
package util;

import java.util.regex.Pattern;

public final class Validador {

    public static final int MIN_CLAVE = 8;
    private static final int MAX_CORREO = 254;
    private static final String[] ETIQUETAS = {"", "Débil", "Aceptable", "Buena", "Fuerte"};

    // local@dominio.tld sin puntos dobles ni guiones al inicio o final de cada etiqueta
    private static final Pattern CORREO = Pattern.compile(
            "^[A-Za-z0-9]([A-Za-z0-9._%+-]*[A-Za-z0-9])?"
            + "@([A-Za-z0-9]([A-Za-z0-9-]*[A-Za-z0-9])?\\.)+[A-Za-z]{2,}$");

    private Validador() {
    }

    public static boolean estaVacio(String texto) {
        return texto == null || texto.isBlank();
    }

    public static boolean correoValido(String correo) {
        return correo != null
                && correo.length() <= MAX_CORREO
                && !correo.contains("..")
                && CORREO.matcher(correo).matches();
    }

    /** Devuelve 0 si no hay texto y de 1 (débil) a 4 (fuerte) en los demás casos. */
    public static int fuerza(String clave) {
        if (clave == null || clave.isEmpty()) {
            return 0;
        }
        if (clave.length() < MIN_CLAVE) {
            return 1;
        }
        boolean mayus = clave.chars().anyMatch(Character::isUpperCase);
```

```
        boolean minus = clave.chars().anyMatch(Character::isLowerCase);
        boolean digito = clave.chars().anyMatch(Character::isDigit);
        boolean simbolo = clave.chars().anyMatch(c -> !Character.isLetterOrDigit(c));

        int puntos = 1;
        if (clave.length() >= 12) puntos++;
        if (mayus && minus && digito) puntos++;
        if (simbolo) puntos++;
        return Math.min(puntos, 4);
    }

    public static String etiquetaFuerza(int nivel) {
        return ETIQUETAS[Math.max(0, Math.min(nivel, ETIQUETAS.length - 1))];
    }
}
```

Practica2/src/vista/FrmOlvido.java

```
package vista;

import dao.UsuarioDAO;
import dao.UsuarioDAOImpl;
import util.Validador;
import vista.ui.Boton;
import vista.ui.CampoClave;
import vista.ui.CampoTexto;
import vista.ui.Enlace;
import vista.ui.MedidorClave;
import vista.ui.PanelMarca;
import vista.ui.Piezas;
import vista.ui.Sello;
import vista.ui.Tema;
import vista.ui.Validable;

import javax.swing.BorderFactory;
import javax.swing.BoxLayout;
import javax.swing.JComponent;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.KeyStroke;
import java.awt.BorderLayout;
import java.awt.CardLayout;
import java.awt.Dimension;
import java.awt.event.KeyEvent;

public class FrmOlvido extends JFrame {

    private static final String PASO_FORMULARIO = "formulario";
    private static final String PASO_LISTO = "listo";

    private final UsuarioDAO usuarioDAO;

    private final CampoTexto txtCorreo = new CampoTexto();
    private final CampoClave txtNueva = new CampoClave();
    private final CampoClave txtConfirmar = new CampoClave();
    private final MedidorClave medidor = new MedidorClave();
    private final JLabel errCorreo = Piezas.mensaje();
    private final JLabel errNueva = Piezas.mensaje();
    private final JLabel errConfirmar = Piezas.mensaje();
    private final Boton btnGuardar = new Boton("Guardar contraseña", true);

    private final CardLayout pasos = new CardLayout();
    private final JPanel lado = new JPanel(pasos);

    public FrmOlvido() {
        this(new UsuarioDAOImpl());
    }

    public FrmOlvido(UsuarioDAO usuarioDAO) {
        this.usuarioDAO = usuarioDAO;
        armarVentana();
        conectarEventos();
    }

    private void armarVentana() {
        setTitle("Recuperar contraseña");
```

```
        setIconImage(Tema.icono());
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);
        setResizable(false);

        lado.setBackground(Tema.PAPEL);
        lado.add(crearFormulario(), PASO_FORMULARIO);
        lado.add(crearConfirmacion(), PASO_LISTO);

        JPanel raiz = new JPanel(new BorderLayout());
        raiz.add(new PanelMarca("Recupere el acceso a su cuenta.",
                "La nueva contraseña se guarda cifrada con SHA-256 y un salt aleatorio, nunca en texto
plano."),
                BorderLayout.WEST);
        raiz.add(lado, BorderLayout.CENTER);
        raiz.setPreferredSize(new Dimension(870, 620));
        setContentPane(raiz);

        pack();
        setLocationRelativeTo(null);
    }

    private JPanel crearFormulario() {
        JPanel p = columna();
        p.add(Piezas.titulo("Nueva contraseña"));
        p.add(Piezas.hueco(8));
        p.add(Piezas.parrafo("Escriba el correo de su cuenta y elija una contraseña nueva."));
        p.add(Piezas.hueco(24));

        p.add(Piezas.etiqueta("Correo electrónico"));
        p.add(Piezas.hueco(6));
        p.add(Piezas.alinear(txtCorreo));
        p.add(errCorreo);
        p.add(Piezas.hueco(10));

        p.add(Piezas.etiqueta("Nueva contraseña"));
        p.add(Piezas.hueco(6));
        p.add(Piezas.alinear(txtNueva));
        p.add(Piezas.hueco(8));
        p.add(Piezas.alinear(medidor));
        p.add(errNueva);
        p.add(Piezas.hueco(10));

        p.add(Piezas.etiqueta("Confirmar contraseña"));
        p.add(Piezas.hueco(6));
        p.add(Piezas.alinear(txtConfirmar));
        p.add(errConfirmar);
        p.add(Piezas.hueco(14));

        p.add(Piezas.alinear(btnGuardar));
        p.add(Piezas.hueco(14));
        Enlace cancelar = new Enlace("Cancelar", this::dispose);
        p.add(cancelar);
        return p;
    }

    private JPanel crearConfirmacion() {
        JPanel p = columna();
        p.add(Piezas.hueco(110));
        p.add(new Sello());
        p.add(Piezas.hueco(26));
        p.add(Piezas.titulo("Contraseña actualizada"));
        p.add(Piezas.hueco(10));
        p.add(Piezas.parrafo("Ya puede iniciar sesión con su nueva contraseña."));
        p.add(Piezas.hueco(34));
        Boton volver = new Boton("Volver al inicio", true);
        volver.addActionListener(e -> dispose());
        p.add(Piezas.alinear(volver));
        return p;
    }

    private static JPanel columna() {
        JPanel p = new JPanel();
        p.setLayout(new BoxLayout(p, BoxLayout.Y_AXIS));
        p.setOpaque(false);
        p.setBorder(BorderFactory.createEmptyBorder(44, 52, 36, 52));
        return p;
    }
```

```
    private void conectarEventos() {
        btnGuardar.addActionListener(e -> guardar());
        getRootPane().setDefaultButton(btnGuardar);
        getRootPane().registerKeyboardAction(e -> dispose(),
                KeyStroke.getKeyStroke(KeyEvent.VK_ESCAPE, 0), JComponent.WHEN_IN_FOCUSED_WINDOW);

        // el error de un campo desaparece apenas el usuario lo corrige
        Piezas.alEscribir(txtCorreo, () -> limpiar(errCorreo, txtCorreo));
        Piezas.alEscribir(txtNueva, () -> {
            limpiar(errNueva, txtNueva);
            medidor.medir(new String(txtNueva.getPassword()));
        });
        Piezas.alEscribir(txtConfirmar, () -> limpiar(errConfirmar, txtConfirmar));
    }

    private void guardar() {
        String correo = txtCorreo.getText().trim();
        String nueva = new String(txtNueva.getPassword());
        String confirmar = new String(txtConfirmar.getPassword());

        if (!hayErrores(correo, nueva, confirmar)) {
            if (usuarioDAO.recuperarContrasena(correo, nueva)) {
                pasos.show(lado, PASO_LISTO);
            } else {
                marcar(errCorreo, txtCorreo, "No hay ninguna cuenta con ese correo.");
                txtCorreo.requestFocusInWindow();
            }
        }
    }

    /** Revisa todo el formulario de una vez y deja el foco en el primer campo con error. */
    private boolean hayErrores(String correo, String nueva, String confirmar) {
        limpiar(errCorreo, txtCorreo);
        limpiar(errNueva, txtNueva);
        limpiar(errConfirmar, txtConfirmar);
        JComponent primero = null;

        if (Validador.estaVacio(correo)) {
            marcar(errCorreo, txtCorreo, "Escriba su correo electrónico.");
            primero = txtCorreo;
        } else if (!Validador.correoValido(correo)) {
            marcar(errCorreo, txtCorreo, "El correo no tiene un formato válido.");
            primero = txtCorreo;
        }

        if (nueva.isEmpty()) {
            marcar(errNueva, txtNueva, "Escriba la nueva contraseña.");
        } else if (Validador.estaVacio(nueva)) {
            marcar(errNueva, txtNueva, "La contraseña no puede ser solo espacios.");
        } else if (nueva.length() < Validador.MIN_CLAVE) {
            marcar(errNueva, txtNueva, "Use al menos " + Validador.MIN_CLAVE + " caracteres.");
        }
        if (primero == null && !errNueva.getText().isBlank()) {
            primero = txtNueva;
        }

        if (confirmar.isEmpty()) {
            marcar(errConfirmar, txtConfirmar, "Repita la contraseña.");
        } else if (!confirmar.equals(nueva)) {
            marcar(errConfirmar, txtConfirmar, "Las contraseñas no coinciden.");
        }
        if (primero == null && !errConfirmar.getText().isBlank()) {
            primero = txtConfirmar;
        }

        if (primero != null) {
            primero.requestFocusInWindow();
        }
        return primero != null;
    }

    private static void marcar(JLabel mensaje, Validable campo, String texto) {
        Piezas.decir(mensaje, texto);
        campo.marcarError(true);
    }

    private static void limpiar(JLabel mensaje, Validable campo) {
        Piezas.decir(mensaje, null);
```

```
        campo.marcarError(false);
    }
}
```